Genesis Protocol: A Lotka–Volterra Inspired Software Organism with Cryptographic Provenance

Kevan Burns

Independent Researcher · FTH Trading, Norcross, GA

kevan.burns@fthtrading.com

ORCID: 0009-0008-8425-939X

Version 1.0 — February 21, 2026 | GitHub Repository

DOI: 10.5281/zenodo.18646886

© 2026 Kevan Burns. Licensed under CC BY 4.0. Source code licensed under MIT.

Abstract

We present Genesis Protocol, a self-sustaining digital organism implemented as a six-crate Rust workspace in which autonomous agents are born with SHA-256–derived genomes, compete for resources in per-niche pools governed by logistic regeneration, face seasonal environmental oscillation and stochastic catastrophe events, and are subject to natural selection pressure. The ecological model draws on Lotka–Volterra resource competition theory, implementing density-dependent foraging, cross-niche interference, and proportional extraction — replacing the winner-take-all dynamics common in multi-agent simulations with biologically plausible population dynamics. The system formalizes its operational requirements as the Software Organism Protocol (SOP-1) and establishes source code integrity through per-crate Merkle trees whose edition root can be anchored on a production blockchain. We validate the architecture through 143 automated tests across 39 source files totaling 10,015 lines of Rust, and demonstrate that the provenance model developed for literary publishing (Burns, 2026) generalizes to executable software systems without modification to its cryptographic infrastructure.

Keywords: *artificial life, evolutionary computation, Lotka–Volterra, multi-agent systems, resource competition, ecological simulation, software organism, Rust, cryptographic provenance, Merkle trees*

Contents

1. Introduction
2. Related Work
3. System Architecture
4. Ecological Model
5. Energy Economics
6. Selection and Reproduction
7. Provenance Model
8. Merkle Tree System

9. Protocol Specification (SOP-1)
10. Evaluation
11. Discussion
12. Conclusion

References · Appendix A: System Invariants · Appendix B: Crate Hash Table · Appendix C: How to Cite

1. Introduction

Artificial life research has a long history of constructing software systems that exhibit emergent behavior through the interaction of simple agents operating under environmental pressure [1, 2]. However, the majority of such systems share two structural weaknesses: their ecological dynamics are unrealistic (often reducing resource competition to winner-take-all allocation), and their provenance — the ability to independently verify that a particular version of the system produced particular results — is unaddressed.

We present Genesis Protocol, a system that addresses both concerns simultaneously. The organism is a Rust workspace of six library crates and one binary that runs as an HTTP server with a background survival loop. Agents are spawned with genomes derived from SHA-256 hashes of unique seed strings, ensuring genuine genetic diversity from the primordial generation. They compete for resources in per-niche pools that regenerate via logistic growth, experience seasonal oscillation and stochastic perturbation, and face natural selection pressure that culls unfit agents while permitting viable ones to replicate.

This paper makes three contributions:

1. **A biologically grounded ecological model** for multi-agent resource competition, implementing Lotka–Volterra dynamics with density-dependent foraging, cross-niche interference coefficients, and proportional extraction that produces realistic population oscillation, resource depletion, and recovery cycles.
2. **The Software Organism Protocol (SOP-1)**, a formal specification extending the Literary Protocol Standard (LPS-1) from literary publishing [3] to executable software systems, defining the ecological, metabolic, selection, and provenance requirements that any conforming implementation must satisfy.
3. **A complete provenance infrastructure** using per-crate Merkle trees, SHA-256 manifests, and on-chain anchoring capability that enables any party to independently verify the integrity of the source code without trusting the author, hosting platform, or any intermediary.

2. Related Work

2.1 Artificial Life and Multi-Agent Systems

The field of artificial life, formalized by Langton [1], explores emergent behavior in computational systems. Tierra [2] demonstrated that digital organisms competing for CPU cycles could exhibit evolutionary dynamics analogous to biological systems. More recent work on multi-agent systems [4] has explored cooperation, competition, and emergent social behavior, but ecological realism — particularly in resource dynamics — has often been sacrificed for computational convenience.

2.2 Lotka–Volterra Resource Competition

The Lotka–Volterra equations [5, 6] provide a mathematical framework for modeling interspecific competition. The discrete-time formulation used in this work models resource dynamics as:

$$dN_i/dt = r_i \cdot N_i \cdot (1 - (N_i + \Sigma\, \alpha_{ij} N_j) / K_i)$$

where N_i is the population of species i, r_i is the intrinsic growth rate, K_i is the carrying capacity, and α_{ij} is the competition coefficient measuring the effect of species j on species i. This framework has been extensively validated in ecological research [7] but has seen limited application in software agent systems.

2.3 Cryptographic Provenance

Content integrity verification using Merkle trees [8] is well established in distributed systems — Git uses Merkle DAGs for repository integrity [9], and IPFS uses them for content-addressed storage [10]. Burns (2026) [3] applied Merkle tree hashing to literary manuscripts, establishing a five-layer provenance stack anchored on a production blockchain. Our work extends this approach from static content (manuscripts) to dynamic executable systems (software organisms).

3. System Architecture

3.1 Design Principles

Biological plausibility. Ecological dynamics must be grounded in established theory (Lotka–Volterra competition, logistic growth, density dependence) rather than ad hoc heuristics.

Economic coherence. Energy (ATP) is a conserved quantity subject to thermodynamic constraints. No agent can create energy from nothing; all income derives from resource extraction or market rewards.

Cryptographic verifiability. Every source file has a SHA-256 hash. Per-crate Merkle roots compose into an edition root that can be independently recomputed.

Formal specification. Operational requirements are codified as SOP-1, enabling independent conforming implementations.

3.2 Crate Architecture

LAYER	CRATE	RESPONSIBILITY	FILES	LINES
1	`genesis-dna`	Genome, identity, traits, lineage, roles, skills	7	918
2	`metabolism`	ATP accounting, ledger, treasury, proof-of-work	6	1,019
3	`ecosystem`	Communication mesh, problem markets, telemetry	8	1,022
4	`evolution`	Mutation engine, selection pressure, gene transfer	5	638
5	`gateway`	World engine, HTTP server, persistence, security	7	5,172

LAYER	CRATE	RESPONSIBILITY	FILES	LINES
6	`apostle`	Outbound intelligence, conversion, targeting	5	535
—	`tests`	Load simulation integration tests	1	511
Total			**39**	**10,015**

Table 1. Crate decomposition with file and line counts.

3.3 Runtime Model

The organism runs as an Axum HTTP server with a background survival loop. Each epoch executes a deterministic six-step cycle:

```
┌──────────────────────────────────────────────────────────┐
│                    EPOCH CYCLE (6 steps)                  │
├──────────────────────────────────────────────────────────┤
│ Step 0: Environment tick (regenerate, seasons, events)    │
│ Step 1: Resource extraction (proportional foraging per niche)│
│ Step 2: Basal metabolism (cost of living)                 │
│ Step 3: Problem market (supplementary income)             │
│ Step 4: Communication (gated broadcasting)                │
│ Step 5: Mutation (environmentally modulated)              │
│ Step 6: Natural selection (fitness culling + replication)  │
└──────────────────────────────────────────────────────────┘
```

Figure 1. Six-step epoch cycle.

4. Ecological Model

4.1 Resource Pools

Each of the five agent roles (Optimizer, Strategist, Communicator, Archivist, Executor) occupies an ecological niche with an independent resource pool. Pools regenerate via logistic growth:

$$R(t+1) = R(t) + r \cdot R(t) \cdot (1 - R(t)/K)$$

where R(t) is the current resource level, r is the per-epoch regeneration rate (default: 0.12), and K is the carrying capacity. Each pool limits extraction to 40% of its current level per epoch, preventing complete depletion:

$$available(t) = 0.4 \cdot R(t)$$

4.2 Seasonal Oscillation

The environment exhibits periodic variation via sinusoidal modulation of carrying capacity:

$$K_{eff}(epoch) = K \cdot (1 + A \cdot sin(2\pi \cdot epoch / T))$$

where A = 0.25 (±25% amplitude) and T = 100 epochs. This produces genuine boom-and-bust cycles: populations expanding during favorable seasons face resource scarcity during unfavorable ones.

4.3 Stochastic Events

Catastrophe (2% probability per epoch): Reduces carrying capacity to 30–60% of normal for 10–20 epochs. Models mass-extinction–scale events testing population resilience.

Resource boom (5% probability per epoch): Spikes pool levels to 90% of capacity. Models windfall events enabling rapid population expansion.

4.4 Density-Dependent Foraging

Per-agent extraction decreases with niche crowding:

$$density_factor = 1 / (1 + n_{niche} \cdot 0.1)$$

A niche with 10 agents yields a density factor of 0.5 — each agent extracts half as much as it would alone. This implements intraspecific competition: conspecifics are the strongest competitors because they exploit identical resources.

4.5 Cross-Niche Competition

Agents in different niches exert a weaker competitive effect via the cross-niche interference coefficient α = 0.15:

$$cross_penalty = 1 - \alpha \cdot (N_{total} - n_{niche}) / N_{total}$$

4.6 Complete Extraction Formula

The per-agent, per-epoch extraction combines all five ecological factors:

$$E_i = f_i \cdot s_i \cdot (R_{pool}/K_{pool}) \cdot density_factor \cdot cross_penalty \cdot \sigma$$

where f_i is agent fitness, s_i is niche skill match, R_{pool}/K_{pool} is pool utilization ratio, and σ = 2.5 is a scale factor calibrated so that an average agent nets positive ATP at equilibrium.

5. Energy Economics

5.1 ATP as Universal Currency

All economic activity is denominated in Adenosine Triphosphate (ATP), an intentional biological metaphor. ATP is the unit of account, medium of exchange, and store of value within the organism.

5.2 Metabolic Cost

Every agent pays a basal metabolic cost of 0.15 ATP per epoch. Balance is clamped at zero (Invariant M-1); the metabolic tick returns actual amount consumed, preventing negative balances.

5.3 Treasury and Redistribution

A central treasury implements five counter-cyclical mechanisms to prevent capital hoarding and maintain economic circulation:

1. **Market Skim** (5%): Applied to all problem-market rewards, funding the treasury reserve.
2. **Balance Decay** (2%/epoch): All agent ATP balances erode by 2% per epoch, creating velocity pressure.
3. **Wealth Tax** (1% above 100 ATP): Marginal tax on balances exceeding 100 ATP, proceeds flow to treasury.
4. **Stipend Distribution**: Underrepresented roles (below 20% of population) receive proportional subsidies.
5. **Crisis Spending**: When population falls below half of carrying capacity, up to 2.0 ATP per agent is injected.
6. **Overflow Redistribution**: If reserves exceed 30% of total supply, half the excess is distributed equally.

Over a 10,000-epoch simulation (Section 10.5), the treasury collected 9,075 ATP and distributed 9,074 ATP, maintaining a mean reserve of only 0.76 ATP.

5.4 Ledger Integrity

The ATP ledger tracks all transactions with typed entries and computes total supply as the sum of all agent balances at query time (Invariant M-2), eliminating accumulation errors.

6. Selection and Reproduction

6.1 Dynamic Population Cap

$$K_{pop} = \lfloor total_capacity / 15 \rfloor, \text{ clamped to } [10, 500]$$

This couples population dynamics to resource availability: catastrophes reduce both carrying capacity and allowable population.

6.2 Fitness-Based Culling

When population exceeds the dynamic cap, the selection engine eliminates agents with the lowest fitness scores. Agents younger than 10 epochs are exempt (Invariant S-2).

6.3 Stasis and Death

Agents at zero ATP enter stasis. After 3 epochs in stasis, they are marked for death (Invariant S-3).

6.4 Replication

Agents above a fitness threshold with sufficient ATP can replicate, producing offspring with mutated genomes. Births per epoch are capped at 3 (MAX_BIRTHS_PER_EPOCH).

6.5 Mutation

Mutation pressure is modulated by environmental stress — higher pressure during harsh seasons increases mutation rates, driving faster adaptation when the environment demands it.

7. Provenance Model

7.1 Extending Literary Provenance to Software

The provenance model adapts the multi-layer architecture from *Deterministic Literary Publishing* [3]. Where the literary protocol hashes 31 text blocks, 5 artifacts, 10 images, and 10 prompts across four Merkle trees, the software protocol hashes 39 source files across 7 crate-level Merkle trees. The cryptographic infrastructure is identical: SHA-256 hashing, binary Merkle tree construction with the odd-leaf-duplicate rule, and a composite edition root suitable for on-chain anchoring.

7.2 Evidence Chain

LAYER	EVIDENCE	RECORD	PURPOSE
1	Local filesystem	OS metadata	First creation timestamps
2	Git commit history	18 commits on GitHub	Continuous authorship timeline
3	Merkle trees	`dist/merkle.json`	Per-crate integrity proofs
4	SHA-256 edition root	`cd78b3be...d2cc3`	Content integrity fingerprint
5	ORCID	`0009-0008-8425-939X`	Author identity record
6	DOI	`10.5281/zenodo.18646886`	Permanent academic identifier

Table 2. Six-layer evidence chain.

7.3 On-Chain Anchoring

PROPERTY	VALUE
Contract	`0x97f456300817eaE3B40E235857b856dfFE8bba90`
Network	Polygon Mainnet (Chain ID 137)
Method	`anchorEdition(editionRoot, ipfsCID)`
Author Identity	`0xB9ffa688A8Bb332221030BbBE46bE5bF03323170`
Author Wallet	`0xC91668184736BF75C4ecE37473D694efb2A43978`
Estimated Cost	< 0.50 USD

Table 3. On-chain anchoring configuration.

8. Merkle Tree System

8.1 Construction Rules

1. **Leaf computation:** h_i = SHA-256($content_i$) where content is raw bytes of each `.rs` source file.
2. **Internal nodes:** h_{parent} = SHA-256($h_{left} \| h_{right}$) where ‖ is string concatenation of lowercase hex digests.
3. **Odd-leaf rule:** If a layer has an odd number of nodes, the last node is duplicated before pairing.
4. **Ordering:** Files within each crate are sorted alphabetically by path.

8.2 Crate Roots

CRATE	ROLE	LEAVES	ROOT (PREFIX)
`genesis-dna`	Genome and Identity	7	`e348c9a9...`
`metabolism`	Energy and Economics	6	`f739fd3a...`
`ecosystem`	Communication and Markets	8	`88b92989...`
`evolution`	Mutation and Selection	5	`fd7f0afb...`
`gateway`	Server and World Engine	7	`9ec9925d...`
`apostle`	Outbound Intelligence	5	`ab727d84...`
`tests`	Integration Tests	1	`6fa77826...`

Table 4. Per-crate Merkle roots.

8.3 Edition Root

The edition root is the SHA-256 hash of the concatenation of all seven crate roots:

$$editionRoot = SHA\text{-}256(r_1 \| r_2 \| r_3 \| r_4 \| r_5 \| r_6 \| r_7)$$

```
cd78b3be6a7f569a8a7a24d49a47ccd01af0fa5ab7c8a04baa6f7ee0367d2cc3
```

9. Protocol Specification (SOP-1)

The Software Organism Protocol v1 (SOP-1) formalizes the requirements demonstrated by the reference implementation. It specifies five functional layers (Genome, Metabolism, Ecosystem, Evolution, Gateway), a state machine lifecycle, ecological model requirements, energy economics constraints, and provenance verification procedures.

9.1 State Machine

$$GENESIS \rightarrow SPAWNING \rightarrow RUNNING \rightarrow STABLE \rightarrow ADAPTING \rightarrow RUNNING \rightarrow \cdots$$

with a terminal EXTINCTION state when population reaches zero.

9.2 Relationship to LPS-1

SOP-1 extends the Literary Protocol Standard (LPS-1) from *Deterministic Literary Publishing* [3]. The key adaptation is replacing the four-tree literary model with a variable-tree software model (one tree per crate), while retaining the same hash construction, edition root computation, and on-chain anchoring methods.

10. Evaluation

10.1 Test Coverage

The system is validated by **143 automated tests** across all six crates with **0 failures**. The test suite includes unit tests for genome construction, ATP accounting, mesh communication, mutation engines, and selection pressure, plus end-to-end load simulation tests that verify multi-epoch population dynamics and resource sustainability.

10.2 Ecological Realism

The ecological model produces dynamics consistent with Lotka–Volterra predictions: population oscillation synchronized with seasons, resource recovery via logistic growth, catastrophe-driven population declines followed by recovery, niche partitioning via density-dependent foraging, and multi-niche coexistence enabled by the low cross-niche coefficient ($\alpha = 0.15$).

10.3 Provenance Verification

The Merkle tree generator (`scripts/merkle.ps1`) computes the edition root from 39 source files, verifies per-file SHA-256 hashes, reconstructs crate Merkle roots, and confirms the composite edition root `cd78b3be...d2cc3`. Execution completes in under 3 seconds on commodity hardware.

10.4 Invariant Compliance

All 14 defined invariants hold for the current edition (see Appendix A).

10.5 Empirical Simulation: 10,000-Epoch Run

A single deterministic simulation of 10,000 epochs was executed from primordial state (20 agents, 50.0 ATP each). The full dataset is available as `dist/simulation-10k.csv`.

10.5.1 Population Dynamics

METRIC	BOOTSTRAP (0–100)	GROWTH (100–500)	EARLY EQUIL. (500–2000)	MID (2000–5000)	LATE (5000–10000)
Mean Population	45.2	51.2	56.8	56.1	57.4
Mean ATP	510.3	450.8	668.1	633.5	697.8

Mean Fitness	0.5224	0.5292	0.5644	0.5708	0.5623

Table 6. Phase-segmented population and economic metrics.

The population converges from 20 to an equilibrium band of 50–60 within 500 epochs. At steady state, mean population is 57.4 with CV 3.7%. Only 17 epochs show population below 40, all during bootstrap.

10.5.2 ATP Economics

METRIC	VALUE
Mean ATP per Agent (equilibrium)	12.15
ATP Supply Range	232.68 – 1013.03
Equilibrium ATP: mean ± σ	697.8 ± 109.4
Treasury collected / distributed	9,075 / 9,074
Mean treasury reserve	0.76 ATP

Table 7. ATP supply and treasury dynamics.

10.5.3 Selection and Fitness

Mean fitness rises from 0.5224 (bootstrap) to 0.5708 (mid-run), a 9.3% improvement through natural selection. Individual peak fitness reached 0.9824 at epoch 4,718. The fitness plateau around 0.56–0.57 suggests selection–mutation equilibrium.

10.5.4 Demographic Turnover

METRIC	VALUE
Total Births / Deaths	902 / 865
Birth-to-Death Ratio	1.04:1
Epochs with Births	674 (6.7%)
Epochs with Deaths	760 (7.6%)
Max Births / Deaths per Epoch	3 / 4

Table 8. Demographic statistics across 10,000 epochs.

10.5.5 Role Distribution

ROLE	MEAN	MIN	MAX
Optimizer	11.6	9	15
Strategist	11.6	9	14

Communicator	11.5	9	14
Archivist	11.1	8	14
Executor	11.6	9	15

Table 9. Role distribution at equilibrium (epochs 5000–10000).

Near-uniform distribution validates the treasury stipend mechanism: underrepresented roles receive proportional subsidies, preventing niche monopolies.

11. Discussion

11.1 Ecological vs. Economic Models

Most multi-agent simulations use economic models for resource allocation. Our ecological approach produces emergent population dynamics that are difficult to achieve with economic mechanisms, but provides weaker guarantees about individual agent optimality.

11.2 Provenance Generalization

The successful extension of literary provenance to software provenance demonstrates that the cryptographic infrastructure is domain-agnostic. Any structured collection of files can be decomposed into category-level Merkle trees and composed into a single edition root.

11.3 Shared Infrastructure

Both Genesis Protocol and the 2,500 Donkeys literary protocol share the same on-chain infrastructure: the LiteraryAnchor contract, the AuthorIdentity contract, and the author wallet. This demonstrates that a single set of smart contracts can serve as provenance infrastructure for an arbitrary number of works across different domains.

11.4 Limitations

Treasury hoarding (v1.0 defect, corrected). In the initial v1.0 release, five of six treasury mechanisms existed as implemented but uncalled code. Only the skim function was wired into the epoch loop, causing ~88% of ATP to accumulate without redistribution. This was identified through external review, diagnosed as dead-code accumulation, and corrected. The 10,000-epoch validation (Section 10.5) confirms the fix. This defect is disclosed as a case study in the importance of integration testing for economic subsystems.

Ecological simplification. Five discrete niches rather than continuous niche space. **Deterministic pseudo-randomness.** Hash-based PRNG favoring reproducibility; ensemble statistics across multiple seeds would strengthen empirical claims. **Single-chain anchoring.** Cross-chain redundancy not yet executed. **Key management.** Single private key without multi-signature support. **Apostle crate maturity.** Conversion tracking infrastructure at proof-of-concept stage.

12. Conclusion

We have presented Genesis Protocol, a six-crate Rust organism implementing Lotka–Volterra inspired resource competition with cryptographic provenance verification. The system demonstrates that biologically grounded ecological dynamics produce realistic population behavior, and that the provenance model established for literary publishing generalizes to executable software.

A 10,000-epoch empirical run validates the model: population converges to a stable equilibrium band (mean 57.4, CV 3.7%), ATP per agent stabilizes at 12.15, mean fitness improves 9.3% through selection, and the treasury cycles 9,074 of 9,075 collected ATP back into the economy. The only significant defect discovered through review (treasury hoarding due to uncalled redistribution code) was corrected and is disclosed transparently in Section 11.4.

The organism comprises 39 source files totaling 10,015 lines of Rust, validated by 143 automated tests. Its integrity is committed through 7 crate-level Merkle trees composing into a single edition root (`cd78b3be...d2cc3`) anchorable on a production blockchain at a cost of less than $0.50 USD.

Every claim in this paper is independently verifiable. The source code is public on GitHub. The Merkle trees are recomputable from the source files. The invariants are testable against the running code. The on-chain infrastructure is source-verified on Polygonscan.

References

[1] C. G. Langton, "Artificial life," in *Artificial Life*, Santa Fe Institute Studies in the Sciences of Complexity, vol. 6, pp. 1–47, Addison-Wesley, 1989.

[2] T. S. Ray, "An approach to the synthesis of life," in *Artificial Life II*, Santa Fe Institute Studies in the Sciences of Complexity, vol. 10, pp. 371–408, Addison-Wesley, 1992.

[3] K. Burns, "Deterministic literary publishing: A multi-layer provenance model for verifiable manuscripts," Independent research, v1.0, Feb. 2026. DOI: 10.5281/zenodo.18646886.

[4] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed., John Wiley & Sons, 2009.

[5] A. J. Lotka, "Contribution to the theory of periodic reactions," *Journal of Physical Chemistry*, vol. 14, no. 3, pp. 271–274, 1910.

[6] V. Volterra, "Variations and fluctuations of the number of individuals in animal species living together," *ICES Journal of Marine Science*, vol. 3, no. 1, pp. 3–51, 1928.

[7] P. Chesson, "Mechanisms of maintenance of species diversity," *Annual Review of Ecology and Systematics*, vol. 31, pp. 343–366, 2000.

[8] R. C. Merkle, "A digital signature based on a conventional encryption function," *Advances in Cryptology — CRYPTO '87*, Lecture Notes in Computer Science, vol. 293, pp. 369–378, Springer, 1988.

[9] S. Chacon and B. Straub, *Pro Git*, 2nd ed., Apress, 2014.

[10] J. Benet, "IPFS — Content addressed, versioned, P2P file system," arXiv:1407.3561, 2014.

[11] Reproducible Builds Project, "Reproducible builds," https://reproducible-builds.org, 2023.

Appendix A: System Invariants

All 14 invariants must hold at all times during system operation.

ID	DOMAIN	INVARIANT	ENFORCEMENT
E-1	Ecology	Logistic Resource Regeneration	$R += r \cdot R \cdot (1 - R/K)$
E-2	Ecology	Seasonal Modulation	$K_{eff} = K \cdot (1 + A \cdot \sin(2\pi \cdot epoch/T))$
E-3	Ecology	Proportional Extraction	Winner-take-all prohibited
E-4	Ecology	Density-Dependent Foraging	$1/(1 + n \cdot 0.1)$
M-1	Metabolism	Non-Negative Balances	Clamp at zero
M-2	Metabolism	Computed Supply	Sum of balances at query time
M-3	Metabolism	Atomic Replication Cost	Single deduction operation
S-1	Selection	Dynamic Population Cap	total_capacity/15, [10,500]
S-2	Selection	Maturation Guard	10-epoch exemption
S-3	Selection	Stasis Tolerance	3-epoch grace period
G-1	Genome	Cryptographic Diversity	SHA-256 derived seeds
G-2	Genome	Responsive Mutation	Seasonal stress modulation
P-1	Provenance	Deterministic Edition Root	Merkle(crate_roots)
P-2	Provenance	Complete Manifest	All 39 files covered

Table A1. Complete invariant registry.

Appendix B: Crate Hash Table

All SHA-256 Merkle roots as of edition v1.0 (git commit `889a560978939d7a40cdf2bebc8c539c0b83b435`).

CRATE	MERKLE ROOT
`genesis-dna`	`e348c9a9678af25a67e7d9c6cfe21702b588abf21415a316ce33b33d9f6f2197`
`metabolism`	`f739fd3aab927ce26847aa6a311ba5d7df64af1c451ce4df331361db4ec5e290`
`ecosystem`	`88b9298b76f6d4e207fb0f55303282ad7421111171a4fbcfd264f74f037e19dc`
`evolution`	`fd7f0afbd6ab83d59eff5624c5144f28e50be1727156e3fd904f5fe90e3ec775`
`gateway`	`9ec9925dc0f2df9d07d73603cdb8a99e0c07356e70a5bd9ef1446501f01732e8`

CRATE	MERKLE ROOT
`apostle`	`ab727d8401a8275836a29309a344021e0414def4cd753b747cdfdf4926e87944`
`tests`	`6fa7782e6959b9ae5bafbf0651109bacf85b46f7070e95d10f39232aaf4926cd`
Edition Root	**`cd78b3be6a7f569a8a7a24d49a47ccd01af0fa5ab7c8a04baa6f7ee0367d2cc3`**

Table B1. Per-crate Merkle roots and composite edition root.

Source Repository: https://github.com/FTHTrading/AI
Author Wallet: `0xC91668184736BF75C4ecE37473D694efb2A43978`
Author Identity Contract: `0xB9ffa688A8Bb332221030BbBE46bE5bF03323170`
LiteraryAnchor Contract: `0x97f456300817eaE3B40E235857b856dfFE8bba90`

Appendix C: How to Cite This Work

BibTeX

```
@techreport{burns2026genesis,
  title     = {Genesis Protocol: A Lotka--Volterra Inspired Software Organism
               with Cryptographic Provenance},
  author    = {Burns, Kevan},
  year      = {2026},
  month     = {February},
  version   = {1.0},
  doi       = {10.5281/zenodo.18646886},
  url       = {https://doi.org/10.5281/zenodo.18646886},
  note      = {Independent research. Reference implementation: 39 Rust source
               files, 10,015 lines, 143 tests. Edition root:
               cd78b3be6a7f569a8a7a24d49a47ccd01af0fa5ab7c8a04baa6f7ee0367d2cc3.}
}
```

APA (7th Edition)

Burns, K. (2026). *Genesis Protocol: A Lotka–Volterra inspired software organism with cryptographic provenance* (Version 1.0). Independent research. https://doi.org/10.5281/zenodo.18646886

Chicago

Burns, Kevan. "Genesis Protocol: A Lotka–Volterra Inspired Software Organism with Cryptographic Provenance." Version 1.0. Independent research, February 2026. https://doi.org/10.5281/zenodo.18646886.

IEEE

K. Burns, "Genesis Protocol: A Lotka–Volterra Inspired Software Organism with Cryptographic Provenance," Independent research, v1.0, Feb. 2026. DOI: 10.5281/zenodo.18646886. [Online]. Available: https://doi.org/10.5281/zenodo.18646886

Version 1.0 — February 21, 2026
Kevan Burns — Independent Researcher — FTH Trading, Norcross, GA
ORCID: 0009-0008-8425-939X
Source code: https://github.com/FTHTrading/AI
Companion work: *Deterministic Literary Publishing* (Burns, 2026) — https://github.com/FTHTrading/2500-donkeys